UNIVERSITY OF LEEDS

COMP1860: Building Our Digital World Computer Systems & Architecture

S1.W5.L2 Hardware Description Language & Computer Aided Design LAB 2

Learning Outcomes

UNIVERSITY OF LEEDS

- Analyse and design a combinational logic circuit
- Utilize a hardware simulator to design, test, and analyze the behavior of a digital circuit
- Implement and debug combinational logic circuits using HDL

Text Book

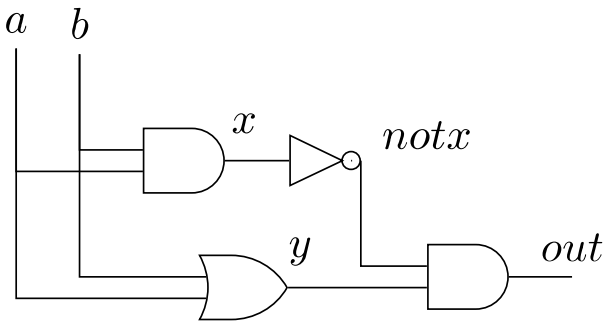
UNIVERSITY OF LEEDS

Nand2Tetris HDL Guide / Documentation:

The Elements of Computing Systems / Appendix 2: HDL

Hardware Description Language

UNIVERSITY OF LEEDS




```
/**
 * Xor gate.
 */

CHIP Xor {
    IN a, b;
    OUT out;

    PARTS:
    And (a=a, b=b, out=x);
    Or (a=a, b=b, out=y);
    Not (in=x, out=notx);
    And (a=y, b=notx, out=out);
}
```

Header

Parts

Nand2Tetris HDL Guide / Documentation:

The Elements of Computing Systems / Appendix 2: HDL

Overview

UNIVERSITY OF LEEDS

- Look at Solution for final problem from lab 1.5.1
- Learn how to implement buses in HDL
- Build some more of the chips for our Hack computer
- Opportunity to start work on Activity sheet 1.6

From last time

UNIVERSITY OF LEEDS

Two binary numbers, a and b, each of two digits, are represented as a_1a_0 and b_1b_0 where a_1, a_0 and b_1, b_0 are Boolean variables which form the digits of the binary numbers.

Devise a combinatorial logic circuit, or circuits to provide outputs of X,Y, Z such that X=1 when a>b, Y=1 when a<b and Z=1 when a=b.

Implement the circuits in HDL. Create an appropriate test script and compare file to test your solution.

a1a0	b1b0	x
00	00	0
00	01	0
00	10	0
00	11	0
01	00	1
01	01	0
01	10	0
01	11	0
10	00	1
10	01	1
10	10	0
10	11	0
11	00	1
11	01	1
11	10	1
11	11	0

		a1a0			
		00	01	11	10
b1b0	00	0	1	1	1
	01	0	0	1	1
	11	0	0	0	0
	10	0	0	1	0

$a1 \wedge \neg b1 \vee a0 \wedge \neg b0 \wedge \neg b1 \vee \neg b0 \wedge a0 \wedge a1$

Multi-bit bus

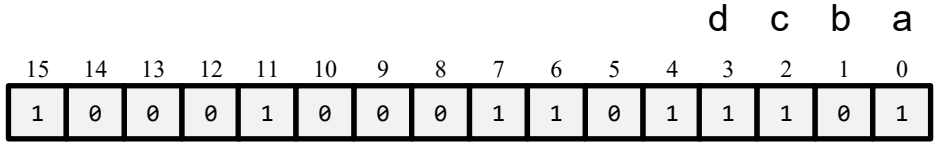
UNIVERSITY OF LEEDS

In HDL sometimes we wish to manipulate a *sequence of bits* as a single entity

将一系列位作为一个整体来操作，看成信号数组即可

In Nand2Tetris such a multi-bit entity is termed “bus”

Example: 16-bit bus



16-bit bus：

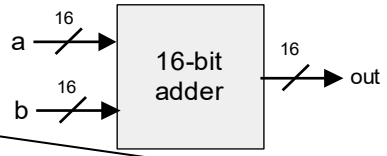
把16个1bit的 输入/输出信号并联在一起，每1bit对应一个输入/输出信号

Working with buses: Example

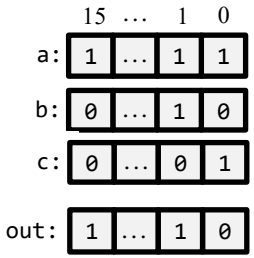
UNIVERSITY OF LEEDS

```
/* Adds two 16-bit values. */
CHIP Adder {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    ...
}
```



[16] 代表16-bits bus，16个1bits的信号



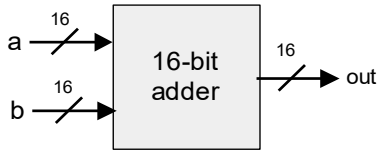

```
/* Adds three 16-bit inputs.
*/
CHIP Adder3Way {
    IN a[16], b[16], c[16];
    OUT out[16];
```

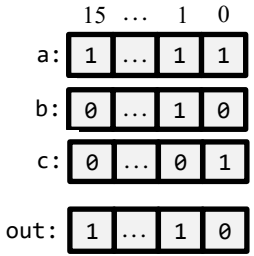
Working with buses: Example

UNIVERSITY OF LEEDS

```
/* Adds two 16-bit values. */
CHIP Adder {
   IN a[16], b[16];
   OUT out[16];

   PARTS:
   ...
}
```






```
/* Adds three 16-bit inputs.
*/
CHIP Adder3Way {
   IN a[16], b[16], c[16];
   OUT out[16];

   PARTS:
   Adder(a=a , b=b, out=ab);
   Adder(a=ab, b=c, out=out);
}
```

定义3个16-bits bus输入a, b, c；和1个16-bits输出out

n-bit value (bus) can be treated as a single entity

Creates an internal bus pin (ab)

16-bits数 a + b + c = out: a + b->ab ; ab + c -> out

Working with individual buses

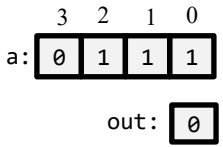
UNIVERSITY OF LEEDS

```
/* Returns 1 if a==1 and b==1,
 * 0 otherwise. */
CHIP And {
    IN a, b;
    OUT out;
    ...
}
```

```
/* 4-way And: Ands 4 bits. */
CHIP And4Way {
    IN a[4];
    OUT out;

    PARTS:
    And(a=a[0],   b=a[1], out=and01);
    And(a=and01,  b=a[2], out=and012);
    And(a=and012, b=a[3], out=out);
}
```

Input bus pins can be subscripted.



要做4个信号输入的and gate，需要把3个2个信号输入的and gate顺序连接起来。

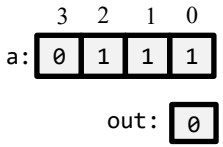
out = a[0] and a[1] and a[2] and a[3]

a是4-bits bus，包含4路信号。a在接and时，需要指定用4-bits信号中的哪一位作为输入，所以用下标指定（和数组一样a[1]）

Working with individual bits within buses

UNIVERSITY OF LEEDS

```
/* Returns 1 if a==1 and b==1,
 * 0 otherwise. */
CHIP And {
   IN a, b;
   OUT out;
   ...
}
```



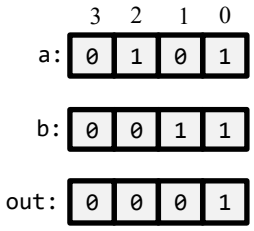

```
/* 4-way And: Ands 4 bits. */
CHIP And4Way {
   IN a[4];
   OUT out;

   PARTS:
   And(a=a[0],   b=a[1], out=and01);
   And(a=and01,  b=a[2], out=and012);
   And(a=and012, b=a[3], out=out);
}
```

Input bus pins can be subscripted.

And4Way： 4Bits输入，1Bit输出

out = a[0] and a[1] and a[2] and a[3]




```
/* Bit-wise And of two 4-bit inputs */
CHIP And4 {
   IN a[4], b[4];
   OUT out[4];

   PARTS:
   And(a=a[0], b=b[0], out=out[0]);
   And(a=a[1], b=b[1], out=out[1]);
   And(a=a[2], b=b[2], out=out[2]);
   And(a=a[3], b=b[3], out=out[3]);
}
```

Output bus pins can be subscripted

And4： 2 * 4Bits输入, 4Bits输出

out[0] = a[0] and b[0]
out[1] = a[1] and b[1]
out[2] = a[2] and b[2]
out[3] = a[3] and b[3]

Bus技巧：将1-bit信号扩展到16-bits bus

UNIVERSITY OF LEEDS

```
CHIP Mux16bits2way {
    IN a[16], b[16],sel;
    OUT out[16];

    PARTS:
        Not16(in[0]=sel,in[1]=sel,in[2]=sel,in[3]=sel,
              in[4]=sel,in[5]=sel,in[6]=sel,in[7]=sel,
              in[8]=sel,in[9]=sel,in[10]=sel,in[11]=sel,
              in[12]=sel,in[13]=sel,in[14]=sel,in[15]=sel,
              out=notSel16 );

        Not16(in=notSel16,out=sel16);
```

Bus技巧：信号切片

UNIVERSITY OF LEEDS

您还可以使用总线中的一系列位，而不是整个总线。

例如，如果您只想使用And16门输出中的4个最低有效位，或者只想使用16位计数器或寄存器的8位。

```
CHIP MyChip {
    IN a[16], b[16];
    OUT out[16];

     PARTS:
     And16(a=a, b=b, out=out, out[0..3]=lsb);
     And4Way(a=lsb, out=4wayout);
}
```

Outputs the whole 16 bit bus

Outputs only bus pins 0 to 7 as an internal bus called lsb of width 4.

Bus技巧：设置全真、全假、部分真假

UNIVERSITY OF LEEDS

You can manually set pins, buses or parts of buses to high (true) or low (false) values. This can be useful for masking bits.
You can also connect different sources to different parts of a bus.

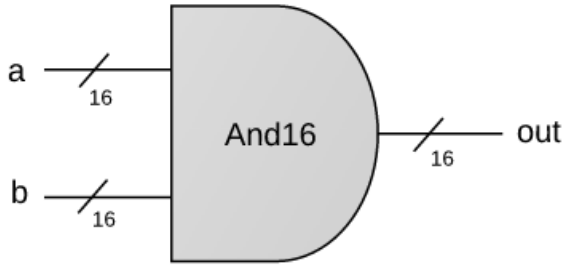
```
CHIP MyChip {
    IN input1[16], input2[16];
    OUT output[16];

     PARTS:
     And16(a=input1, b=true, out=output);
     And16(a[0..7]=false, a[8..15]=input1[8..15], out=output);
}
```

Sets all 16 bits of input b to true.

将a的8个最低有效位设置为false，将输入1的8个最高有效位设置成最高有效位。

Your Turn: And16



Example:

```
a = 1 0 1 0 1 0 1 1 0 1 0 1 1 1 0 0
b = 0 0 1 0 1 1 0 1 0 0 1 0 1 0 1 0
-----------------------------------
out = 0 0 1 0 1 0 0 1 0 0 0 0 1 0 0 0
```

```
CHIP And16 {
    IN a[16], b[16];
    OUT out[16];

     PARTS:
     // Put your code here:
}
```

Implementation tip

A straightforward 16-bit extension of the elementary `And` gate

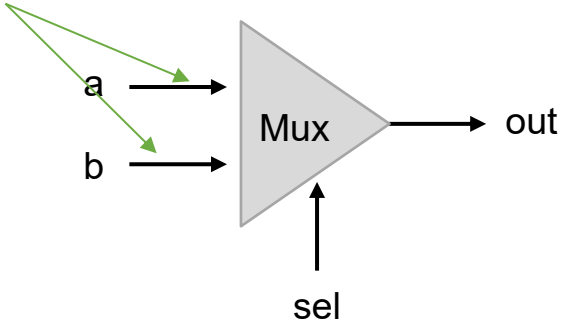
(See notes about working with *multi-bit buses*).

Your turn: Implement a 2-Way 1-bit multiplexer (MUX)

UNIVERSITY OF LEEDS

Your saw Multiplexers in lecture 1.4.1

- A multiplexer is a selector
- One of the inputs will be output
- Which one depends on the selection input (sel)
- The preamble for the chip is:

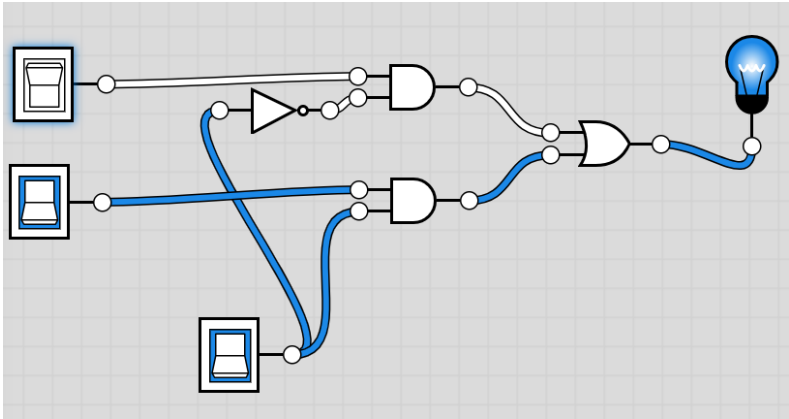


sel	out
0	a
1	b

Mux.hdl

```
CHIP Mux {
    IN a, b, sel;
    OUT out;

    PARTS:
    // Put your code here:
}
```



2-Way 1-bit multiplexer (MUX)

UNIVERSITY OF LEEDS

2-way: 2路输入信号　　1-bit：每路输入信号带宽1-bit

```
CHIP Mux2way {

   IN a, b, sel;

   OUT out;

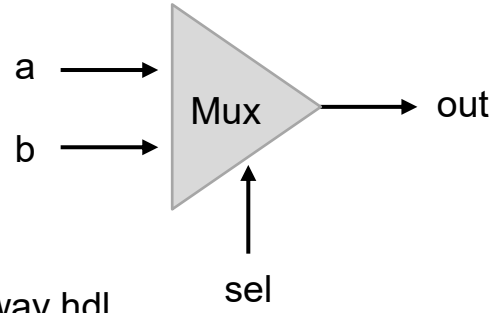

   PARTS:

   Not(in=sel, out=notSel);

   And (a=a, b=notSel, out=outa);

   And (a=b, b=sel, out=outb);

   Or  (a=outa, b=outb, out=out);
}
```




```
load          Mux2way.hdl,
output-file Mux2way.out,
output-list  a b sel out;
```

```
set a 0, set b 1, set sel 0, eval, output;
set a 0, set b 1, set sel 1, eval, output;
set a 1, set b 0, set sel 0, eval, output;
set a 1, set b 0, set sel 1, eval, output;
```

Your turn: Implement a 2-Way 16-bit multiplexer

UNIVERSITY OF LEEDS

In our computer we are likely to want to select between 16-values rather than 1-bit values.

- Create a new chip Mux16 that selects between 16-bit input values and outputs a 16-bit output.

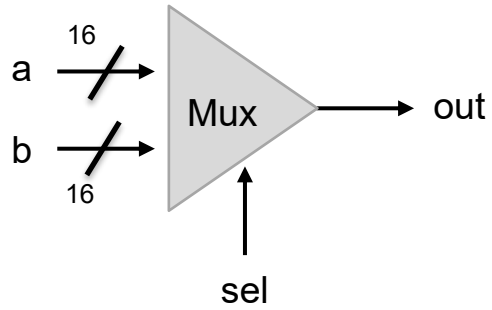
Mux16.hdl

```
CHIP Mux16 {
    IN a[16], b[16], sel;
    OUT out[16];

    PARTS:
    // Put your code here:
}
```

The Nand2Tetris built in chips provides the following chips that you might find helpful:

- Not16(in,out) a 16-bit wide Not gate.
- Or16(a,b,out) a 16-bit wide Or Gate.



2-Way 16-bit multiplexer

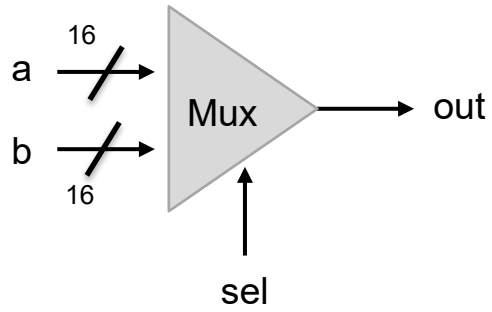
UNIVERSITY OF LEEDS

2-way: 2路输入信号　　16-bits：每路输入信号带宽16-bits

```
CHIP Mux2way16 {
    IN      a[16], b[16], s;
    OUT     out[16];

    PARTS:
        // 把1-bits sel信号变成16-bits sel信号
        Not16(in[0]=s, in[1]=s, in[2]=s, in[3]=s,
              in[4]=s, in[5]=s, in[6]=s, in[7]=s,
              in[8]=s, in[9]=s, in[10]=s, in[11]=s,
              in[12]=s, in[13]=s, in[14]=s, in[15]=s,
              out=notS16 );

        Not16(in = notSel16, out=sel16);
        And16(a=a, b=notSel16,  out= outA16 );
        And16(a=b, b=sel16,       out= outB16 );
        Or16(a=outA16, b=outB16, out=out);
}
```



load Mux2way16.hdl,
output-file Mux2way16.out,
output-list a b sel out;

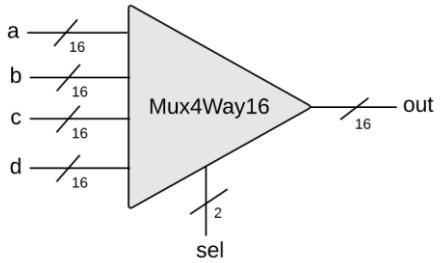
set a 255, set b 65280, set s 0, eval, output;
set a 255, set b 65280, set s 1, eval, output;

Your turn: Implement a 16-bit, 4-way multiplexer

UNIVERSITY OF LEEDS

As you saw in lecture 1.4.1 you can also build multiplexers that select between more than 2 inputs.

On the right is an example of a 4-way (4-to-1) Multiplexer.



sel[1]	sel[0]	out
0	0	**a**
0	1	**b**
1	0	**c**
1	1	**d**

`Mux4Way16.hdl`

```
CHIP Mux4Way16 {
    IN a[16], b[16], c[16], d[16],
       sel[2];
    OUT out[16];

    PARTS:
    // Put your code here:
}
```

Implementation tip:

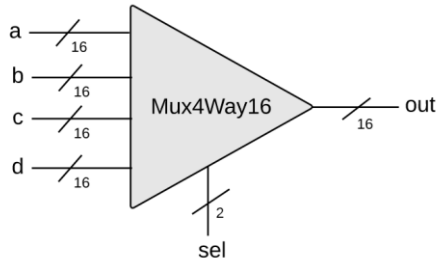
We can use a 2-bit bus for the selection bits to provide 4 possible values for sel..

The chip can be built from several `Mux16` gates.

16-bit, 4-way multiplexer

UNIVERSITY OF LEEDS

4-way: 4路输入信号　　16-bits： 每路输入信号带宽16-bits




```
CHIP Mux4way16 {
   IN    a[16], b[16],c[16],d[16],sel[2];
   OUT  out[16]
   PARTS:
/*  sel[1]是高位，sel[0]是低位     a = 00  b = 01  c = 10  d = 11
     简单说： sel[1]=1,选出cd； sel[1]=0,选出ab； 然后再用selp[0]选

     sel[1]先在00,10中选出一个放到outx0
     sel[1]再在01,11中选出一个放到out1x
     sel[0]在outx0和out1x中选一个，为0:选outx0；否则out1x */
    Mux16(a=a,b=c,sel=sel[1],out=outx0);
    Mux16(a=b,b=d,sel=sel[1],out=out1x);
    Mux16(a=outx0,b=out1x,sel=sel[0],out=out);
}
```

load Mux4way16.hdl,
output-file Mux4way16.out,
output-list a b c d sel out;

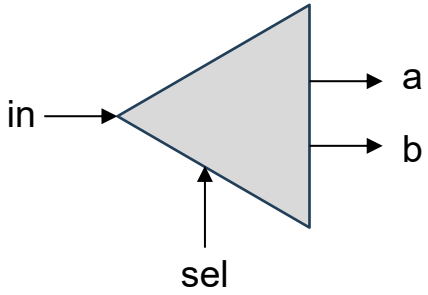
set a 1, set b 2, set c 3, set d 4;
set sel 0, eval, output;
set sel 1, eval, output;
set sel 2, eval, output;
set sel 3, eval, output;

Your turn: Implement a 2-way demultiplexer (DMUX)

UNIVERSITY OF LEEDS

As you saw in lecture 1.4.1 Demultiplexers work the other way. They select the output path

- The sel input determines which output line (a or b) the input (in) is sent to.



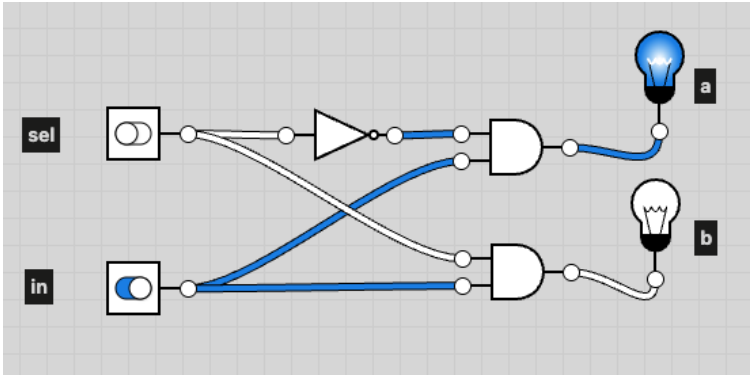
sel	Use output
0	**a**
1	**b**

DMux.hdl

```
CHIP DMux {

    IN in, sel;
    OUT a, b;

    PARTS:
            // Put your code here:
}
```

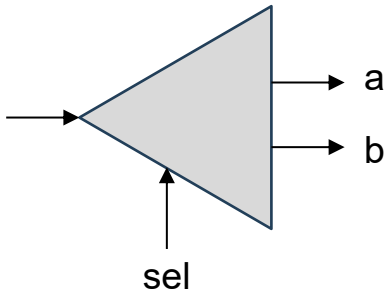


2-way 1-bit demultiplexer (DMUX)

UNIVERSITY OF LEEDS

2-way: 2路输入信号　　1-bit： 每路输入信号带宽1-bit

```
CHIP Demux2way {
   IN in,sel;
   OUT a,b;
   PARTS:
   /*
      1. 先把控制信号Sel反一下，形成Sel/notSel两个控制信号
      2. and (notSel信号, a )
      3. and (sel信号, a )
   */
    Not(in= sel, out= notSel);
    And(a=in , b= notSel, out= a);
    And(a=in , b= sel, out= b);
}
```



load Demux2way.hdl,
output-file Demux2way.out,
output-list in sel a b;

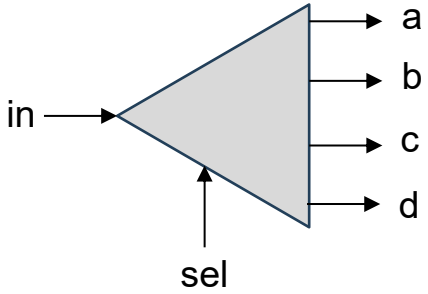
set in 1, set sel 0, eval, output;
set in 1, set sel 1, eval, output;

Your turn: A 4-way demultiplexer

UNIVERSITY OF LEEDS

We can expand this to have as many outputs as we wish.

We can use a bus for the selection bits to provide all possible values for sel.



sel[1]	sel[0]	out
0	0	**a**
0	1	**b**
1	0	**c**
1	1	**d**

`DMux4Way.hdl`

```
CHIP DMux4Way {

    IN in, sel[2];
    OUT a, b, c, d;

    PARTS:
            // Put your code here:
}
```

Implementation tip:

The chip can be built from several `DMux` gates.

4-way 1-bit demultiplexer

UNIVERSITY OF LEEDS

4-way: 4路输入信号　　1-bit: 每路输入信号带宽1-bit

```
CHIP Demux4way {

   IN in, sel[2];

   OUT a, b, c, d;


   PARTS:

   DMux(in=in , sel=sel[1] , a= out0x, b= out1x);

   DMux(in=in , sel=sel[0] , a= outx0, b= outx1 );

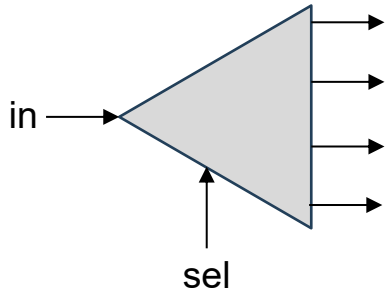

   And(a= out0x, b= outx0, out=a );

   And(a= out0x, b= outx1, out=b );

   And(a= out1x, b= outx0, out=c );

   And(a= out1x, b= outx1, out=d );

}
```



load Demux4way.hdl,
output-file Demux4way.out,
output-list in sel a b c d;

set in 1;
set sel 0, eval, output;
set sel 1, eval, output;
set sel 2, eval, output;
set sel 3, eval, output;

Your turn

UNIVERSITY OF LEEDS

- Now have a go at Activity Sheet 1.5
  - Request the activity sheet using the link in Minerva (question 2 is part of your portfolio)
  - You should upload the tst files obtained from the activity sheet to your repository to help you test your solutions to question 2.
  - Once you have a working solution for question 2, submit your solution to Gradescope to see if it passes all the tests. You can do this by uploading your A6Q2.hdl file.



Your turn:

UNIVERSITY OF LEEDS

If you wish to finish building some other gates we will use then complete the list below

Given: `Nand`

Goal: Build the following gates:

Elementary logic gates

- `Not`
- `And`
- `Or`
- `Xor` ✓
- `Mux` ✓
- `DMux` ✓

16-bit variants

- `Not16`
- `And16` ✓
- `Or16`
- `Mux16` ✓

Multi-way variants

- `Or8Way`
- `Mux4Way16` ✓
- `Mux8Way16`
- `DMux4Way` ✓
- `DMux8Way`

Summary

UNIVERSITY OF LEEDS

- Learnt how to implement buses in HDL
- Started to build the chips we will need for our Hack computer

  And16
  Mux
  Mux16
  Mux4Way16